

Aufgabenblatt 1

Java: Einführung, Objektorientierte Programmierung

Wichtige Ankündigungen

- Das Vorlesungsmaterial, die Übungsblätter und die Vorlagen für die Hausaufgaben finden Sie unter <https://git.tu-berlin.de/algodat-sose25/material.git>.
- Alle Hausaufgaben sind in Einzelarbeit zu erledigen. Kopieren Sie niemals Code und geben Sie Code in keiner Form weiter.
- Wenn Ihre Abgabe nicht im richtigen Ordner liegt, nicht kompiliert, unerlaubte packages oder imports enthält oder zu spät abgegeben wird, gibt es **0 Punkte** auf diese Abgabe.
- Es gibt drei Aufgabentypen:
 - **Tutorium** → Besprechung im Tutorium
 - **Hausaufgabe** → Eigenarbeit mit Hilfe in Rechnerübungen
 - **Klausurvorbereitung** → Optionale Klausurübung

Abgabe (bis 19.05.2025 23:59 Uhr)

Die folgenden Dateien müssen für eine erfolgreiche Abgabe im git Ordner eingecheckt sein:

Geforderte Dateien:

Blatt01/src/Pair.java	Aufgabe 2
Blatt01/src/Polygon.java	Aufgabe 3
Blatt01/src/ConvexPolygon.java	Aufgabe 3
Blatt01/src/Tetragon.java	Aufgabe 3
Blatt01/src/Triangle.java	Aufgabe 3
Blatt01/src/RegularPolygon.java	Aufgabe 3

Als Abgabe wird jeweils nur die letzte Version im main branch in git gewertet.

Aufgabe 1: Objektorientierte Programmierung (Tutorium)

Diese Aufgabe soll Ihnen die Grundprinzipien der Objektorientierten Programmierung näher bringen, welche Sie auch in der Hausaufgabe brauchen. Die Aufgabe wird im Tutorium gemeinsam erarbeitet. Alternativ (oder zusätzlich) können Sie sich das Videotutorium anschauen und dabei die Aufgabenteile selbstständig lösen, indem Sie die Videos pausieren. Sollte Ihnen das noch schwer fallen, schauen Sie sich die Videos komplett an und bearbeiten Sie die Aufgabenteile danach alleine, indem Sie umsetzen, was Sie gerade gelernt haben.

1.1 Definieren Sie Klassen und Objekte.

Lösung

- eine **Klasse** ist der Bauplan, das Grundkonstrukt auf dem alles aufbaut. Zum Beispiel ein Autobauplan. Dieser enthält alle Infos über das Auto (Attribute, aber auch eine Bauanleitung), aber ist selbst noch kein Auto.
- Das **Objekt** ist eine Instanz einer Klasse, also in diesem Fall das Auto, das durch den Bauplan gebaut wurde.

1.2 Implementieren Sie eine Klasse Kegelrobbe und testen Sie Ihre Implementation in einer main-Methode.

Lösung

Eine beispielhafte Implementierung mit einigen Attributen und Methoden:

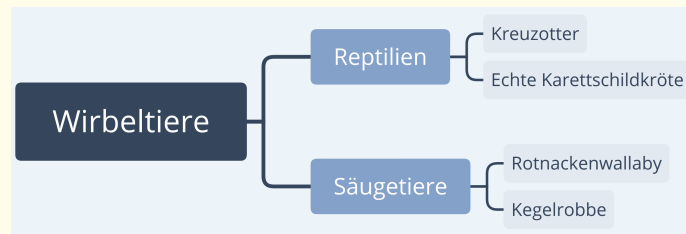
```
// Hier beginnt die Definition der Klasse "Kegelrobbe"
public class Kegelrobbe {
    // Attribute (Eigenschaften) werden definiert
    private String name;
    private int alter;
    private double groesse;

    // Konstruktor.
    // Wird verwendet um eine Instanz der Klasse zu erstellen
    public Kegelrobbe(String name, int alter, double groesse){
        this.name = name;
        this.alter = alter;
        this.groesse = groesse;
    }

    // eine "getter"-Methode, die den Namen zurückgibt.
    // Auf den Namen kann nicht direkt zugegriffen werden,
    // weil er "private" ist
    public String getName(){
        return name;
    }
}
```

- 1.3 Erstellen Sie eine Hierarchie von *Wirbeltieren* am Beispiel von zwei Klassen von Wirbeltieren mit jeweils zwei Vertretern dieser Klassen.

Lösung



- 1.4 Wenn Sie nun diese Hierarchie implementieren mit *Wirbeltiere* als abstrakte Klasse. Überlegen Sie sich Attribute, die Sie den Klassen geben könnten. Wie sähe der Code dafür aus, wenn Sie in jeder Klasse nur den Konstruktor implementieren? Welche dieser Attribute werden unter welchen Bedingungen vererbt?

Lösung

Beispielhafte Klassen:
Wirbeltiere.java

```
public abstract class Wirbeltiere{
    private int alter;
    public Wirbeltiere(int alter){
        this.alter = alter;
    }
}
```

Saeugetiere.java

```
public abstract class Saeugetiere extends Wirbeltiere{
    public Saeugetiere(int alter){
        super(alter);
    }
}
```

Kegelrobbe.java

```
public class Kegelrobbe extends Saeugetiere{
    public Kegelrobbe(int alter){
        super(alter);
    }
}
```

- 1.5 Wie testen Sie die Funktionalität Ihrer Klassen? Wie erstellen Sie einen Array von *Wirbeltiere*? Können Sie unterschiedliche Tiere in diesem Array speichern? Wenn ja, wie? Wie funktioniert

Casting an diesem Beispiel?

Lösung

```
Wirbeltiere [] V = new Wirbeltiere[3];  
V[0] = new Kreuzotter();  
V[1] = new Rotnack.wallaby();  
V[2] = new Kegelrobbe();  
Kreuzotter K = (Kreuzotter) V[0];
```

1.6 Was ist Polymorphie? Implementieren Sie eine abstrakte Methode `essen()`.

Lösung

Polymorphie bedeutet „Mehrgestaltigkeit“. Das heißt in diesem Kontext, dass eine Referenz vom gleichen Typ auf verschiedene Klassen zeigen kann. Wenn dann Methoden dieser Klasse aufgerufen werden, wird die Methode der tatsächlichen Klasse aufgerufen, nicht die des Referenztyps. Abstrakte Methoden **müssen** von erbbenden Klassen implementiert werden, außer sie sind selbst wieder abstrakt.

```
// Datei Wirbeltiere.java  
public abstract class Wirbeltiere{  
    public abstract void essen();  
}  
  
// Datei Saeugetiere.java  
public abstract class Saeugetiere extends Wirbeltiere{  
}  
  
// Datei Kegelrobbe.java  
public class Kegelrobbe extends Saeugetiere{  
    @Override  
    public void essen(){  
        System.out.println("Mmm, lecker Fisch");  
    }  
}  
  
// Datei Rotnack.wallaby.java  
public class Rotnack.wallaby extends Saeugetiere{  
    @Override  
    public void essen(){  
        System.out.println("Mmm, lecker Pflanzen");  
    }  
    public void main(){  
        Wirbeltiere k = new Kegelrobbe();  
        Wirbeltiere r = new Rotnack.wallaby();  
        k.essen(); // Ausgabe: Mmm, lecker Fisch  
        r.essen(); // Ausgabe: Mmm, lecker Pflanzen  
    }  
}
```

Aufgabe 2: Für die Hausaufgaben wird keine Lösung bereitgestellt.

Aufgabe 3: Für die Hausaufgaben wird keine Lösung bereitgestellt.

Aufgabe 4: Laufzeitorientierung (Klausurvorbereitung)

Das Two-Sum-Problem ist ein typisches Problem, das Ihnen z.B. in einem Vorstellungsgespräch gestellt werden könnte:

Es sind ein Array und eine Zahl k gegeben, beide vom Typ `int`. Schreiben Sie eine Methode, welche prüft, ob es in dem Array ein Paar gibt, welches in der Summe k ergibt.

4.1 Geben Sie je ein Beispiellarray an, für den diese Methode für $k = 3$ `true` oder `false` ausgibt.

Lösung

true: `A=[1,2,3]`, weil $1 + 2 = 3$

false: `A=[1,3,4]`, weil $1 + 3 \neq 3$ und $1 + 4 \neq 3$ und $3 + 4 \neq 3$

4.2 Geben Sie die Brute-force Lösung des Problems und deren Laufzeit an.

Lösung

```
public boolean sumsToTarget(int[] arr, int k) {  
    for (int i = 0; i < arr.length; i++) {  
        for (int j = i + 1; j < arr.length; j++) {  
            if (arr[i] + arr[j] == k) {  
                return true;  
            }  
        }  
    }  
    return false;  
}
```

Die Laufzeit beträgt $O(n^2)$.

4.3 Überlegen Sie sich eine schnellere Lösung, indem Sie den Array sortieren und dann von vorne und hinten gleichzeitig anfangen.

Lösung

```
public boolean sumsToTarget(int[] arr, int k) {
    Arrays.sort(arr);
    int lhs = 0;
    int rhs = arr.length - 1;
    while (lhs < rhs) {
        int sum = arr[lhs] + arr[rhs];
        if (sum == k) {
            return true;
        } else if (sum < k) {
            lhs++;
        } else {
            rhs--;
        }
    }
    return false;
}
```

Je nach Sortieralgorithmus unterschiedlich, aber hier $O(n \log(n))$.

Quelle: https://web.stanford.edu/class/cs9/sample_probs/TwoSum.pdf

Die Lösung wird in dem Videotutorium 1 als Video bereitgestellt. Versuchen Sie erst selbst auf die Lösung zu kommen, aber mindestens sie danach selbst zu rekonstruieren.

Was Sie nach diesem Blatt wissen sollten:

- was Objekte und Klassen sind.
- wie man Konstruktoren schreibt, Objekte erzeugt und Methoden auf diesen Objekten aufruft.
- wie Sie for-Schleifen und if-Bedingungen verwenden.
- was getter- und setter-Methoden sind und wie man sie schreibt.
- wie Sie Arrays von Objekten erstellen und über diese iterieren.
- was eine abstrakte Klasse ist und wie Klassen erben.
- wie Sie eine `main()`-Methode schreiben und ausführen.
- wie Sie Ihre geschriebenen Methoden selbst ausprobieren und Kontrollvariablen ausgeben.